Design Analysis for GridEnv Reinforcement Learning Project

---

1. Environment Design

State Representation

The environment was designed as an (M X N) discrete grid world where the agent, obstacles, and goal occupy distinct cells. The state is represented as a 2D NumPy array (or flattened 1D array for the learning algorithm) with integer encodings:

- **0:** Empty cell
- **1:** Obstacle
- **2:** Goal
- **3:** Agent

This representation is compact, human-interpretable, and compatible with both convolutional and dense neural network architectures. Flattening the 2D grid into a 1D vector allows use of standard MLP-based policies in Stable-Baselines3 without additional feature engineering.

Reward Function

The reward structure was designed to guide the agent toward the goal while penalizing inefficient or unsafe behavior: - **Goal reached:** `+2.0`
- **Collision (obstacle/boundary):** `-1.0`
- **Step penalty:** `-0.05`
- **Shaping term:** `+0.2 * (old_distance - new_distance)`

This hybrid reward scheme provides dense feedback, allowing the agent to learn from intermediate progress rather than waiting for sparse terminal signals. The shaping term encourages the agent to minimize Manhattan distance to the goal, promoting directional exploration. Early experiments with higher penalties and sparse rewards led to poor convergence, as the agent often preferred inaction over exploration.

These design decisions make the environment learnable with standard RL algorithms while keeping the challenge of obstacle avoidance intact.

---

2. Algorithm Comparison

Two algorithms from Stable-Baselines3 were evaluated: - **PPO (Proximal Policy Optimization)** — an on-policy algorithm. - **DQN (Deep Q-Network)** — an off-policy algorithm.

Results Summary (after convergence)

Algorithm	Success Rate (%)	Avg. Steps	Avg. Reward
DQN	**25.0**	**15.26**	**-0.436**
PPO	**15.0**	**8.43**	**-0.718**

Analysis

DQN outperformed PPO across all metrics except Avg. Steps, achieving a higher success rate and better average reward. This difference can be attributed to the following factors:

- **Off-policy learning advantage:** DQN reuses past experiences through a replay buffer, making it more sample-efficient in environments with discrete state spaces and deterministic dynamics.
- **Exploration strategy:** DQN's ε-greedy exploration ensures broader state coverage, while PPO's stochastic policy often gets trapped in local optima early in training.
- **Reward stability:** DQN better handles moderate negative rewards, whereas PPO's gradient updates can become unstable when the signal is small or noisy.

Despite PPO's robustness in continuous or stochastic environments, its on-policy nature limits efficiency in this grid-based deterministic setup.

---

3. Challenges and Solutions

Challenge 1: Reward Shaping
Initially, the agent struggled to reach the goal due to sparse rewards. Early versions provided only terminal signals, leading to aimless wandering. The introduction of the distance-based shaping term (+0.2 * Δdistance) provided a gradient for learning progress, significantly improving performance.

Challenge 2: Stable Training and Convergence
PPO was particularly sensitive to learning rates and step penalties. The model often diverged or plateaued with inappropriate settings. Reducing the learning rate (1e-4) and

increasing the number of timesteps (100k–300k) improved stability. Monitoring TensorBoard metrics (ep_rew_mean, ep_len_mean) helped detect instability early.

Challenge 3: Low Success Rate
Even after stabilization, success rates remained below 30%. The problem stems from limited feedback when the agent nears obstacles or takes indirect routes. Adjusting reward magnitudes and limiting episode length to 50 steps improved consistency but capped exploration depth. Future work may include curriculum learning (gradually increasing obstacle density) to boost generalization.

---

4. Summary

The project successfully implemented a configurable 2D grid environment for reinforcement learning, trained PPO and DQN agents, and evaluated their performance. While DQN demonstrated better learning efficiency and goal-reaching performance, both algorithms benefited from well-designed reward shaping and stable environment structure. The main lessons learned highlight the sensitivity of RL algorithms to reward design and the importance of iterative tuning for convergence and performance stability.